Clasificación de calidad de vino tinto mediante regresión logística y PyTorch

Fabricio Mena Mejía
Instituto Tecnológico de Costa Rica
Email: fabriciomena11@estudiantec.cr

Anthony Fuentes Calvo
Instituto Tecnológico de Costa Rica
Email: anthonyfuentescalvo@estudiantec.cr

Fernando Sánchez Hidalgo
Instituto Tecnológico de Costa Rica
Email: Fer.sanchez@estudiantec.cr

***Abstract*—El propósito de esta tarea es implementar y analizar un modelo de regresión logística para resolver un problema de clasificación binario para determinar si la calidad de un vino es "BUENA" o "MALA", utilizando como entorno de desarrollo PyTorch para la optimización y creación de modelos personalizados. Se espera que el estudiante realice un análisis exploratorio detallado de las características del conjunto de datos y la variable a predecir, que implemente stratified sampling, ajuste un modelo de regresión logística y evalúe aplicando métricas.**

I. Desarrollo de Regresión Logística con PyTorch

Para este trabajo se utiliza el conjunto de datos *winequality-red* del repositorio UCI Machine Learning (también disponible en Kaggle) [3], que contiene 1 599 observaciones de vinos tintos descritos por 11 características fisicoquímicas numéricas y una variable de calidad sensorial `quality`, con valores enteros entre 3 y 8. El objetivo es construir un modelo de regresión logística binaria que clasifique cada vino como **BUENO** o **MALO**.

Se define una variable objetivo binaria y_{bin} a partir de la calidad original,

$$y_{\text{bin}} = \begin{cases} 1, & \text{si } \texttt{quality} \geq 7 \\ 0, & \text{en otro caso,} \end{cases} \tag{1}$$

de modo que la clase positiva representa vinos de buena calidad (puntuación 7 u 8). La distribución resultante es desbalanceada, con aproximadamente un 13.6 % de vinos de buena calidad y 86.4 % de vinos catalogados como "normales". Esta proporción se mantiene en todas las particiones mediante muestreo estratificado.

Antes de ajustar el modelo se realiza un tratamiento de outliers sobre las seis características seleccionadas ya que algunos boxplots como el de sulfatos y el de la densidad (ver Fig. 1 y Fig. 2). En el cuaderno se aplica el criterio del *rango intercuartílico* (IQR): para cada variable se calculan el primer cuartil Q_1 y el tercer cuartil Q_3, y se eliminan las observaciones con valores fuera del intervalo $[Q_1 - 1.5\,IQR,\, Q_3 + 1.5\,IQR]$, donde $IQR = Q_3 - Q_1$. Esto reduce ligeramente el número de filas pero mejora la robustez del ajuste frente a valores extremos detectados en los boxplots.

Posteriormente, en el cuaderno `Tarea2.ipynb` se divide el conjunto de datos limpio en tres subconjuntos mutuamente

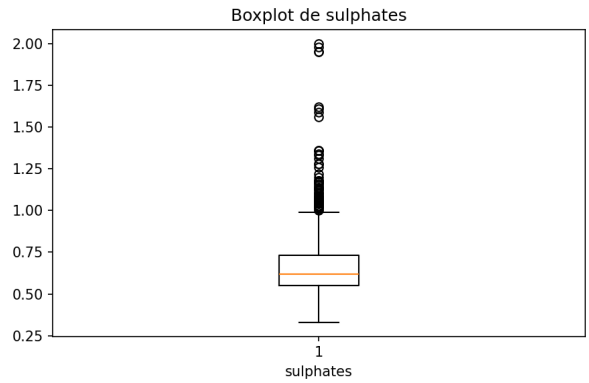

Fig. 1. Boxplot de Sulfatos mostrando outliers.

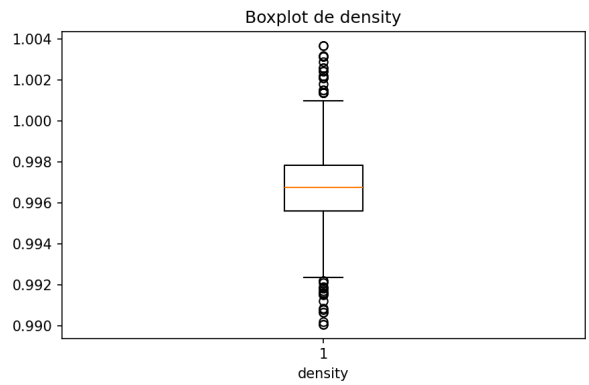

Fig. 2. Boxplot de Densidad mostrando outliers.

excluyentes: 70 % para entrenamiento, 15 % para validación y 15 % para prueba, utilizando `train_test_split` con la opción `stratify` para mantener la proporción de la clase positiva en cada partición. A continuación se escalan las seis características seleccionadas mediante *StandardScaler* (ajustado con los datos de entrenamiento) y se construyen tensores de PyTorch para alimentar el modelo.

Para garantizar la reproducibilidad de los resultados, en el cuaderno se fijan las semillas de los generadores pseudoaleatorios de Python, NumPy y PyTorch (por ejemplo, con una semilla global igual a 42). De este modo, cada vez que se ejecuta el cuaderno completo se obtienen las mismas

particiones y el mismo comportamiento de entrenamiento para las 10 configuraciones de hiperparámetros, haciendo que las métricas reportadas sean deterministas.

El clasificador se implementa como una regresión logística clásica en PyTorch (celda 9). El modelo consta de una única capa lineal con dimensión de entrada igual al número de características seleccionadas,

$$z = w^\top x + b, \qquad \hat{y} = \sigma(z), \tag{2}$$

donde $\sigma(\cdot)$ es la función sigmoide. Para el entrenamiento se utiliza la función de pérdida *Binary Cross-Entropy* con logits `BCEWithLogitsLoss` y el optimizador Adam. En cada época se calcula la pérdida tanto en el conjunto de entrenamiento como en el de validación, lo que permite analizar gráficamente el comportamiento de *underfitting* u *overfitting* del modelo.

II. Justificación de selección de features

El conjunto original de 11 características se reduce a un máximo de seis para evitar modelos innecesariamente complejos y cumplir con la consigna. La selección se basa en dos criterios matemáticos: (i) la correlación lineal con la variable de calidad y (ii) la multicolinealidad entre predictores.

En primer lugar, se calcula la matriz de correlación de Pearson entre todas las variables numéricas (celda 2 del notebook). La Fig. 3 muestra el mapa de calor de dichas correlaciones, donde puede observarse que el grado alcohólico, la acidez volátil, los sulfatos, el ácido cítrico, el dióxido de azufre total y la densidad son las variables con mayor correlación (en módulo) con la calidad.

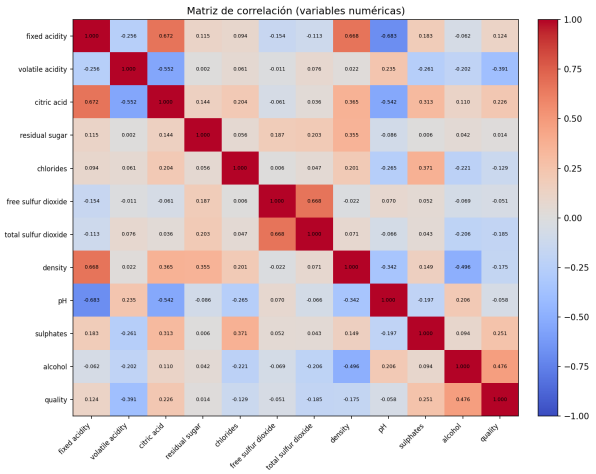

Fig. 3. Matriz de correlación de las variables numéricas del conjunto de vinos tintos.

Sea x_j cada característica numérica y y la calidad original. La correlación de Pearson se define como

$$r(x_j, y) = \frac{\operatorname{cov}(x_j, y)}{\sigma_{x_j}\, \sigma_y}, \tag{3}$$

por lo que $|r(x_j, y)|$ mide la fuerza de la relación lineal entre x_j y la calidad. En la celda 2 se ordenan las características según el valor absoluto de esta correlación. A continuación, en la celda 3 se recorre esa lista ordenada y se construye un conjunto de características S de tamaño máximo 6, aplicando el siguiente procedimiento:

1) Inicialmente $S = \varnothing$.
2) Para cada nueva característica x_j en el orden dado, se calcula su correlación con las ya incluidas en S.
3) Si para alguna $s \in S$ se cumple $|r(x_j, s)| > 0.8$, se considera que hay multicolinealidad severa y se descarta x_j.
4) En caso contrario, se añade x_j a S.

Este algoritmo garantiza que todas las variables de S estén fuertemente relacionadas con la calidad pero no excesivamente correlacionadas entre sí. El conjunto final seleccionado es

$$\begin{aligned} S = \{&\texttt{alcohol},\ \texttt{volatile acidity},\\ &\texttt{sulphates},\ \texttt{citric acid},\\ &\texttt{total sulfur dioxide},\ \texttt{density}\} \end{aligned} \tag{4}$$

Para estas seis características se generan histogramas, boxplots y diagramas de dispersión frente a la calidad original. En los histogramas de alcohol(Fig. 4) y acidez volátil(Fig. 5) podemos ver que la mayor frecuencia de su aparición es en el lado izquierdo, indicando que sus concentraciones suelen ser menores para la mayoría de vinos.

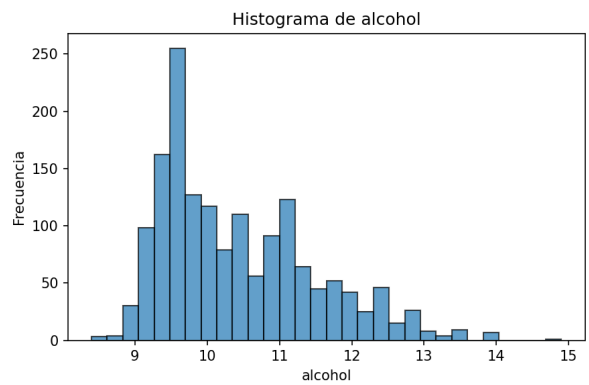

Fig. 4. Histograma de frecuencia vs cantidad de alcohol.

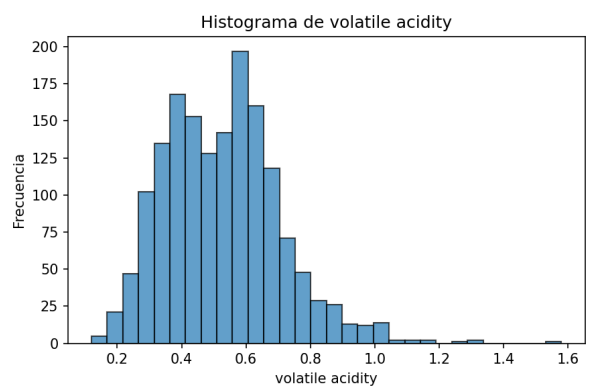

Fig. 5. Histograma de frecuencia vs acidez volátil.

Analizando la Fig. 6 podemos ver que a medida que aumenta el grado de alcohol los puntos tienden hacia la derecha indicando un aumento la calidad del vino; por otro lado en la Fig. 7 se puede interpretar que a medida que la acidez volátil disminuye la calidad del vino aumenta, esto se puede apreciar al ver que la distribución de puntos tiende a la izquierda.

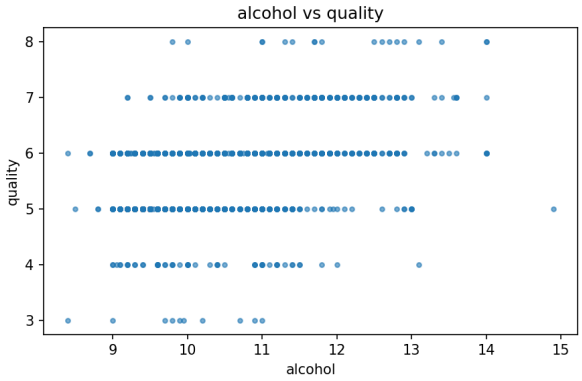

Fig. 6. Diagrama de dispersión de alcohol vs calidad.

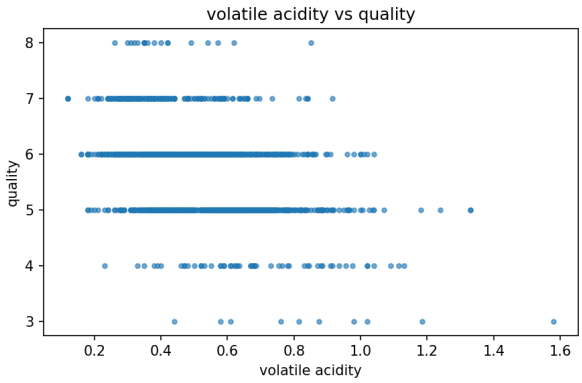

Fig. 7. Diagrama de dispersión de acidez volátil vs calidad.

III. Presentación de métricas por experimento

Se diseñó una rejilla de 10 configuraciones de hiperparámetros para la regresión logística, variando la tasa de aprendizaje, el tamaño de lote y el número de épocas de entrenamiento. Para cada configuración se entrena un modelo independiente y se registran:

- la pérdida de entrenamiento y de validación en cada época;
- las métricas finales en validación: accuracy, precisión, recall, F1-Score y AUC.

Las curvas *Training vs Validation Loss* de los 10 experimentos se generan en la celda 11. Una de ellas, correspondiente al experimento 5, que obtuvo el mejor F1-Score en validación (con `lr = 0.001`, `batch = 64` y 50 épocas), puede verse en la Fig. 8.

En todas las configuraciones la pérdida de entrenamiento y la de validación disminuyen de forma monótona y se

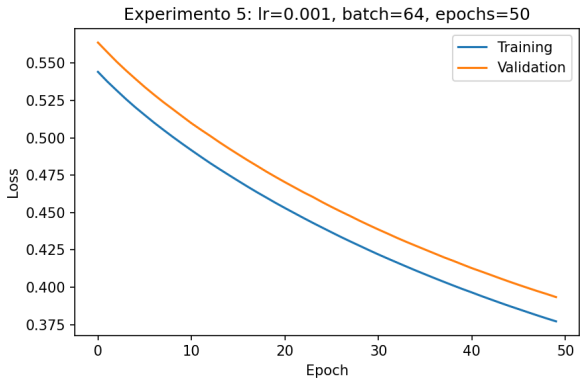

Fig. 8. Curvas de pérdida de entrenamiento y validación para el experimento con `lr = 0.001`, `batch = 64` y 50 épocas.

mantienen cercanas entre sí. No se observa que la pérdida de validación aumente mientras la de entrenamiento sigue decreciendo, lo que indica tanto un buen aprendizaje del modelo como la ausencia de overfitting. En los experimentos con más épocas y tasa de aprendizaje moderada la separación entre ambas curvas es algo mayor, pero sigue siendo razonable, véase en la Fig. 9.

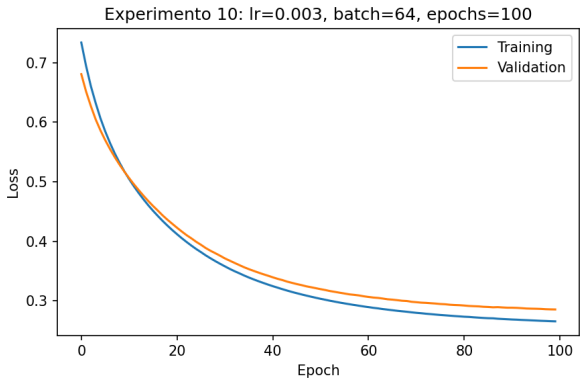

Fig. 9. Curvas de pérdida de entrenamiento y validación para el experimento con `lr = 0.003`, `batch = 64` y 100 épocas.

Las métricas de validación de los 10 experimentos se resumen en una tabla generada también en la celda 11 (dataframe `results_df`). Con el tratamiento de outliers y la partición 70/15/15, la accuracy en validación se sitúa aproximadamente entre 0.80 y 0.89, mientras que el F1-Score oscila entre 0.30 y 0.59. El AUC se mantiene en torno a valores entre 0.78 y 0.88, lo que indica una capacidad de discriminación razonable para distinguir vinos de buena calidad.

El experimento con mejor F1-Score en validación corresponde ahora a la configuración `lr = 0.001`, `batch = 64` y 50 épocas. Este modelo se selecciona como candidato final para ser evaluado en el conjunto de prueba.

IV. Análisis, comparación de resultados y modelo final

Una vez identificado el mejor experimento en validación, se evalúa el modelo resultante sobre el conjunto de prueba, manteniendo los mismos seis predictores y el mismo umbral de decisión 0.5 sobre la probabilidad predicha. Con la partición 70/15/15 y tras eliminar outliers, las métricas obtenidas en test para el modelo final son, de forma aproximada, las siguientes:

- Accuracy ≈ 0.85;
- Precisión ≈ 0.45 sobre la clase "BUENA";
- Recall ≈ 0.52;
- F1-Score ≈ 0.48;
- AUC ≈ 0.80.

Comparando estas métricas con las obtenidas en el conjunto de validación para el mismo experimento (accuracy ≈ 0.89, F1-Score ≈ 0.60, AUC ≈ 0.88), se observa una ligera caída de rendimiento al pasar a prueba, especialmente en F1 y AUC. No obstante, las diferencias no son extremas y el comportamiento sigue siendo coherente con las curvas de pérdida, donde no se aprecia un crecimiento sostenido de la pérdida de validación respecto a la de entrenamiento. Para todos los experimentos se utiliza un umbral de decisión del 0.5, pero si se toma en cuenta que las clases están desbalanceadas, sería interesante repetir los experimentos para analizar y comparar los resultados con un umbral de decisión menor (como por ejemplo 0.4 o 0.3).

Para analizar con mayor detalle los errores del modelo, se genera en la celda 11 la matriz de confusión sobre el conjunto de prueba. La Fig. 10 muestra dicha matriz, donde la clase 1 corresponde a vinos de buena calidad y la clase 0 a vinos normales.

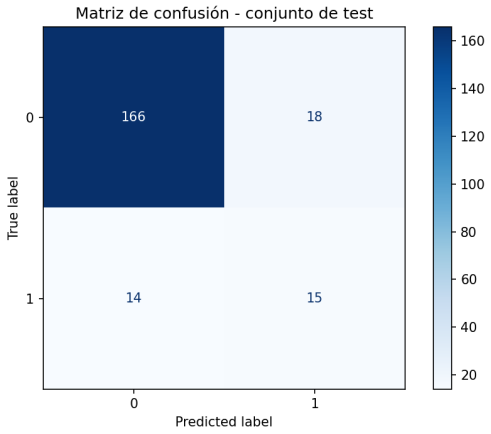

Fig. 10. Matriz de confusión del modelo final evaluado en el conjunto de prueba.

La matriz de confusión evidencia que el modelo acierta la mayoría de los vinos de clase 0 (baja tasa de falsos positivos) y recupera aproximadamente la mitad de los vinos de buena calidad (recall cercano al 0.5). Dado que la clase positiva es minoritaria, este compromiso entre precisión y recall resulta razonable para un modelo lineal sencillo como la regresión logística. Dependiendo de la aplicación, podría ajustarse el umbral de decisión para privilegiar un mayor recall (aceptando más falsos positivos) o una mayor precisión (aceptando perder algunos vinos de buena calidad).

En resumen, el modelo de regresión logística entrenado con las seis características seleccionadas y tras el filtrado de outliers logra un desempeño razonable en validación y prueba, sin signos claros de sobreajuste y con una capacidad moderada para identificar vinos de buena calidad a partir de sus mediciones fisicoquímicas.

A. Tablas de resultados experimentales

La Tabla I resume las métricas de validación obtenidas en los 10 experimentos realizados con distintas combinaciones de tasa de aprendizaje, tamaño de lote y número de épocas. Estos valores provienen del dataframe `results_df` generado en la celda 11 del notebook.

TABLE I
Métricas de validación por experimento de entrenamiento (accuracy, precisión, recall, F1 y AUC).

Exp.	lr	batch	ep.	Acc	Prec	Rec	F1	AUC
1	0.0001	32	50	0.83019	0.42553	0.68966	0.52632	0.85397
2	0.0003	32	50	0.81132	0.35135	0.44828	0.39394	0.78444
3	0.0010	32	50	0.86321	0.50000	0.34483	0.40816	0.88016
4	0.0030	32	50	0.86792	0.54545	0.20690	0.30000	0.88204
5	0.0010	64	50	0.88679	0.58621	0.58621	0.58621	0.87771
6	0.0030	64	50	0.85849	0.47368	0.31034	0.37500	0.87959
7	0.0010	128	50	0.79717	0.37037	0.68966	0.48193	0.87564
8	0.0030	128	50	0.87264	0.56250	0.31034	0.40000	0.87601
9	0.0010	64	100	0.88208	0.56667	0.58621	0.57627	0.87940
10	0.0030	64	100	0.86792	0.54545	0.20690	0.30000	0.88110

Además, en la Tabla II se compara el rendimiento del mejor experimento según el F1-Score en validación (configuración 5 con `lr = 0.001`, `batch = 64` y `50 épocas`) frente a las métricas obtenidas al evaluar ese mismo modelo sobre el conjunto de prueba. Los valores de validación corresponden al diccionario `best_by_f1`, mientras que los de prueba proceden de las variables `test_accuracy`, `test_precision`, `test_recall`, `test_f1` y `test_auc` calculadas en la celda 11. Es importante notar que para este dataset, las clases se encuentran desbalanceadas, esto significa que la métrica del accuracy puede ser afectada por un alto nivel de predicciones en clases negativas, además que se puede identificar que para todos los experimentos, el valor de AUC se mantiene siempre por encima de 0.78, tomando variaciones pequeñas entre valores, por lo que se le otorga la mayor relevancia a los valores de F1-score para definir el mejor modelo.

TABLE II
Comparación de métricas entre validación y prueba para el modelo seleccionado.

Métrica	Validación	Prueba
Accuracy	0.887	0.850
Precisión	0.586	0.455
Recall	0.586	0.517
F1-Score	0.586	0.484
AUC	0.878	0.804

Se observa que las métricas en el conjunto de prueba son ligeramente inferiores a las de validación, especialmente en

términos de F1-Score y AUC. Aun así, la caída de rendimiento es moderada y coherente con las curvas de pérdida, por lo que no se aprecia un sobreajuste extremo y el modelo mantiene una capacidad de generalización razonable sobre datos no vistos.

V. Referencias

References

[1] Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.

[2] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: data mining, inference, and prediction* (2nd ed.). Springer.

[3] Cortez, P., Cerdeira, A., Almeida, F., Matos, T., & Reis, J. (2009). Modeling wine preferences by data mining from physicochemical properties. *Decision Support Systems*, 47(4), 547–553. Dataset UCI: *Wine Quality*.